V. Sarcar, *Java Design Patterns*

https://doi.org/10.1007/978-1-4842-7971-7_6

6. Builder Pattern

Vaskaran Sarcar[1]

(1) Garia, Kolkata, India

This chapter covers the Builder pattern.

GoF Definition

It separates the construction of a complex object from its representation so that the same construction processes can create different representations.

Concept

The Builder pattern is useful for creating complex objects that have multiple parts. The object creation process should be independent of these parts. In addition, you should be able to use the same construction process to create different representations of the objects. This pattern is one of those design patterns that are relatively tough to understand on the very first attempt. Once you see the code and analyze the structure, they become easy. So, keep reading.

Before you move forward, let me show you a code snippet and analyze the potential difficulties. Here is a sample class `A` with a constructor that has many parameters:

```
Class A{
  A(int arg1, int arg2, double arg3,int arg4,B b, C c){
        // Some code to initialize
   }
        // Remaining code is skipped
}
```

Now go through the following points:

- You need to supply a `B` class object and a `C` class object. So, to construct an `A` class instance, you need to create a `B` class object and a `C` class object before you call the `A` class constructor. So, you consider the B object and the C object as parts of an A object. If you need to make a final object from various parts, the Builder pattern is a good choice.
- You can see that you also need to supply three integer arguments. Since these arguments are similar, understanding and using them can be challenging if you do not follow proper naming (for example, you need to remember their order of appearance). But even proper naming won't help if you see a compiled version of the code. In that case, to understand them, you need to refer to the code documentation. This is why passing too many arguments in a method or constructor is not a recommended practice in general. It is better if you pass a minimum number of arguments (preferably one or zero) at a time. The Builder pattern can help you write better code in a similar case.

As per the GoF book, four different players are involved in this pattern. They have the relationships shown in Figure **6-1**.

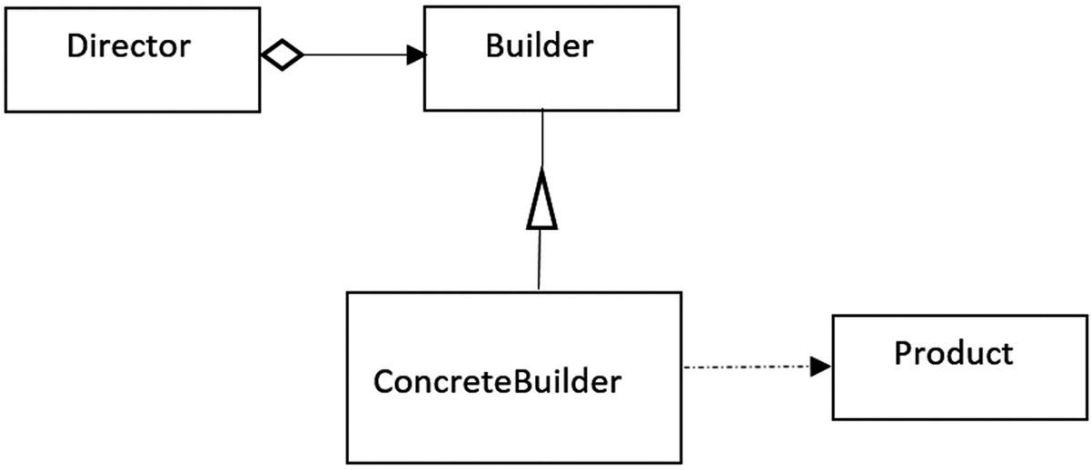


Figure 6-1 A sample of the Builder pattern

Here, the `Product` is the complex object under consideration, and it is the final outcome. In the upcoming examples, vehicles are the products. The `Car` and `MotorCycle` classes are used to make the concrete products, cars and motorcycles, respectively. These classes inherit from the `Vehicle` class.

The `Builder` is an interface that contains the methods to build different parts of a product. The `ConcreteBuilder` implements the `Builder` interface and assembles these parts. In other words, the `ConcreteBuilder` object builds the internal representations of a `Vehicle` instance, and it can have a method that can be called to get this `Vehicle` instance. (As said before, vehicles are the products in the upcoming examples.)

The `Director` is responsible for creating the final object using the `Builder` interface. It is important to note that the `Director` decides the sequence of steps to build the product. You can safely assume that the `Director` can vary the sequence to make different products.

Real-Life Example

To complete an order for a computer, different hardware parts are assembled based on customer preferences. For example, one customer can opt for a 1000GB hard disk with an Intel processor, and another customer can choose a 500GB hard disk with an AMD processor. Here the computer is the final `Product`, the customer plays the role of the `Director`, and the seller/assembler plays the role of the `ConcreteBuilder`.

Computer World Example

The classical GoF book considers an example when a typical application tries to convert one text format to another text format, such as converting from Rich Text Format (RTF) to ASCII. Typically, there is a reader and a converter. The reader parses the document and converts the document to the target format using a converter. This converter has specialized subclasses that perform different types of conversions and assemble the object using an abstract interface. In this example, the converter plays the role of a *builder* and the reader plays the role of a *director*. Here are a few more examples:

- The `Java.util.Calendar.Builder` class is an example in this category. But it is available from `Java 8` onwards only.
- You can also consider the `java.lang.StringBuilder` class as a close example in this context. But you need to remember that the GoF definition also says that when using this pattern, you can use the same con-

struction process to make different representations. In this context, this example does not fully qualify for the GoF's definition.

Implementation

In demonstration 1, `Builder` is used for the `Builder` interface and `CarBuilder` and `MotorCycleBuilder` are two `ConcreteBuilder`s. `Car` and `MotorCycle` are the concrete products. The `Director` class has its usual meaning: it instructs a builder to make a product. In demonstration 1, `CarDirector` and `MotorCycleDirector` are the concrete directors that inherit from the abstract class `Director`. Let's examine them one by one.

Start with the `Builder` interface, which defines the possible methods to build different parts for a vehicle:

```
interface Builder {
     void addBrandName();
     void buildBody();
     void insertWheels();
     // The following method is used to
     // retrieve the object that is constructed.
     Vehicle getVehicle();
}
```

`CarBuilder` and `MotorCycleBuilder` implement this interface. They provide the implementation as per their needs. Here is a sample:

```
// The CarBuilder builds cars.
class CarBuilder implements Builder {
     Car car;
     public CarBuilder() {
          car=new Car("Ford");
     }
     @Override
     public void addBrandName() {
          car.add(" Adding the car brand: " + car.brandName);
     }
     @Override
     public void buildBody() {
          car.add(" Making the car body.");
     }
```

```
        @Override
         public void insertWheels() {
             car.add(" 4 wheels are added to the car.");
        }
        @Override
        public Vehicle getVehicle() {
             return car;
        }
   }
```

`CarDirector` and the `MotorCycleDirector` inherit from the abstract class `Director`. They provide the step-by-step process for building a vehicle. Here is a sample with supporting comments:

```
   // Director class
   abstract class Director {
        // Director knows how to use/instruct the
        // builder to create a vehicle.
        public abstract Vehicle instruct(Builder builder);
   }
   /**
    * The CarDirector directs the
    * car's instantiation steps.
    */
   class CarDirector extends Director {
        // The car director follows
        // its own sequence:
        // Make body-> add wheels->then add the brand name.
        public Vehicle instruct(Builder builder) {
             builder.buildBody();
             builder.insertWheels();
             builder.addBrandName();
             return builder.getVehicle();
        }
   }
```

Inside the client code, you make one car and one motorcycle. Notice that you create the builder object before a `Director` object uses it. The `Director` instance invokes the `instruct(...)` method to instruct a builder object on how to assemble a product. Here is the sample for making a car:

```
   // Making a car
   Builder builder = new CarBuilder();
```

```
Director director = new CarDirector();
Vehicle vehicle=director.instruct(builder);
vehicle.showProduct();
```

The `Vehicle` is an abstract class that is easy to understand. Notice the use of a linked list data structure to combine different parts of a vehicle. Here is its definition:

```
abstract class Vehicle {
     /*
      * You can use any data structure that you prefer.
      * I have used LinkedList<String> in this case.
      */
     private LinkedList<String> parts;
     public Vehicle() {
          parts = new LinkedList<String>();
     }
     public void add(String part) {
          // Adding parts
          parts.addLast(part);
     }
     public void showProduct() {
          System.out.println("These are the construction sequences:")
          for (String part : parts)
               System.out.println(part);
     }
}
```

The `Car` and `MortorCycle` classes extend this `Vehicle` class and are used to get a final product such as a car or a motorcycle.

Now you can go through the complete demonstration.

Class Diagram

Figure **6-2** shows the class diagram.

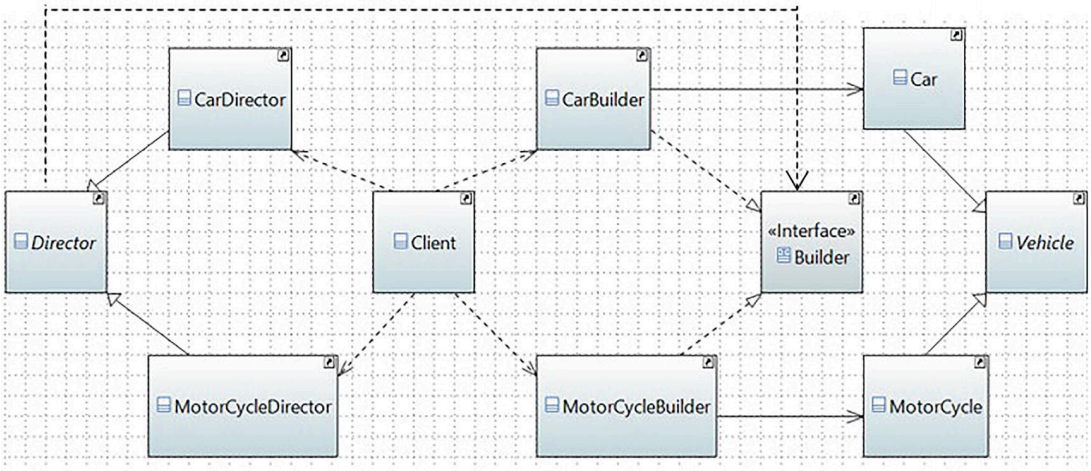
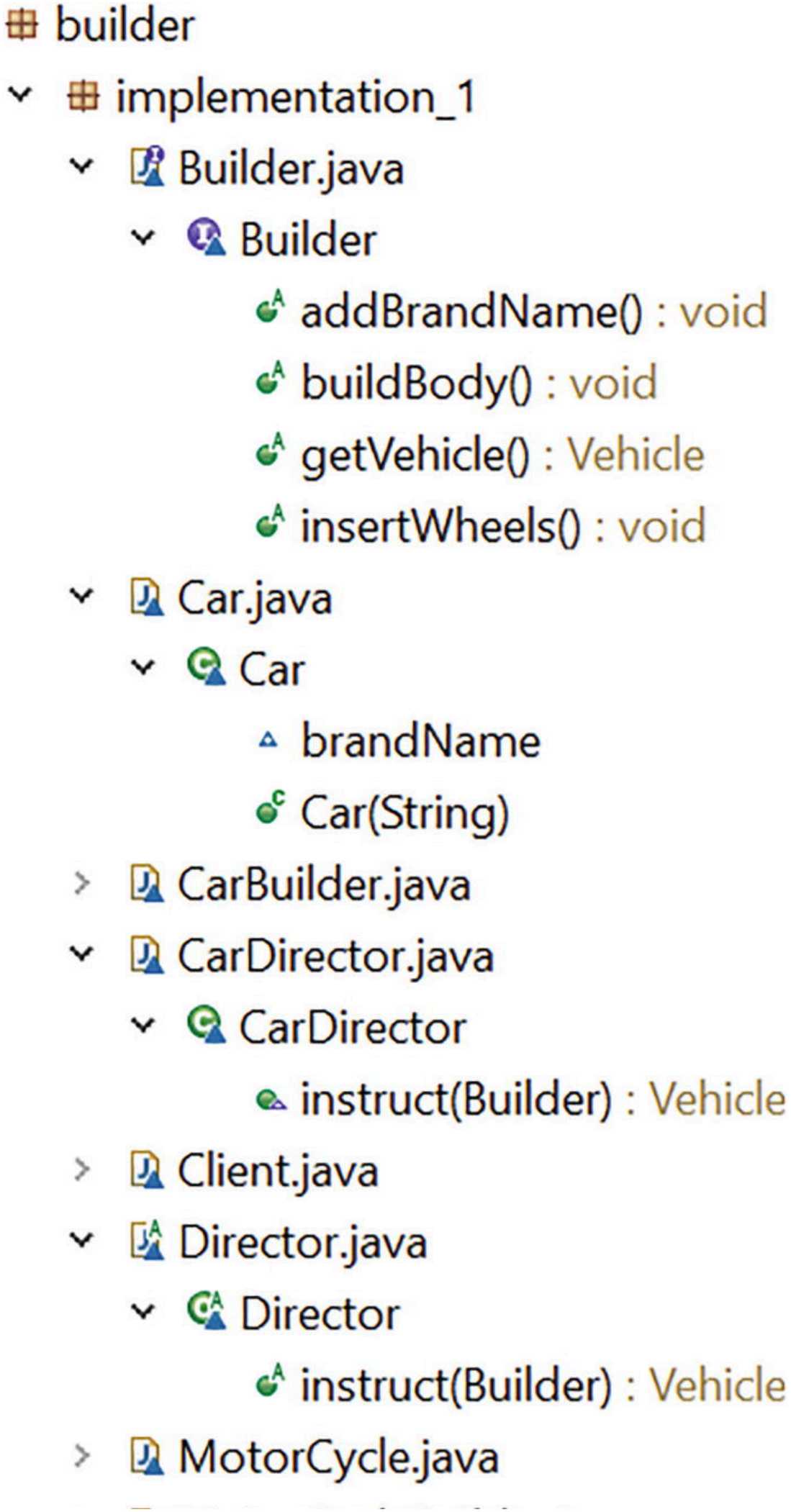


Figure 6-2 Class diagram

Package Explorer View

Figure **6-3** shows the high-level structure of the program.


builder
implementation_1
Builder.java
Builder
addBrandName() : void
buildBody() : void
getVehicle() : Vehicle
insertWheels() : void
Car.java
Car
brandName
Car(String)
CarBuilder.java
CarDirector.java
CarDirector
instruct(Builder) : Vehicle
Client.java
Director.java
Director
instruct(Builder) : Vehicle
MotorCycle.java

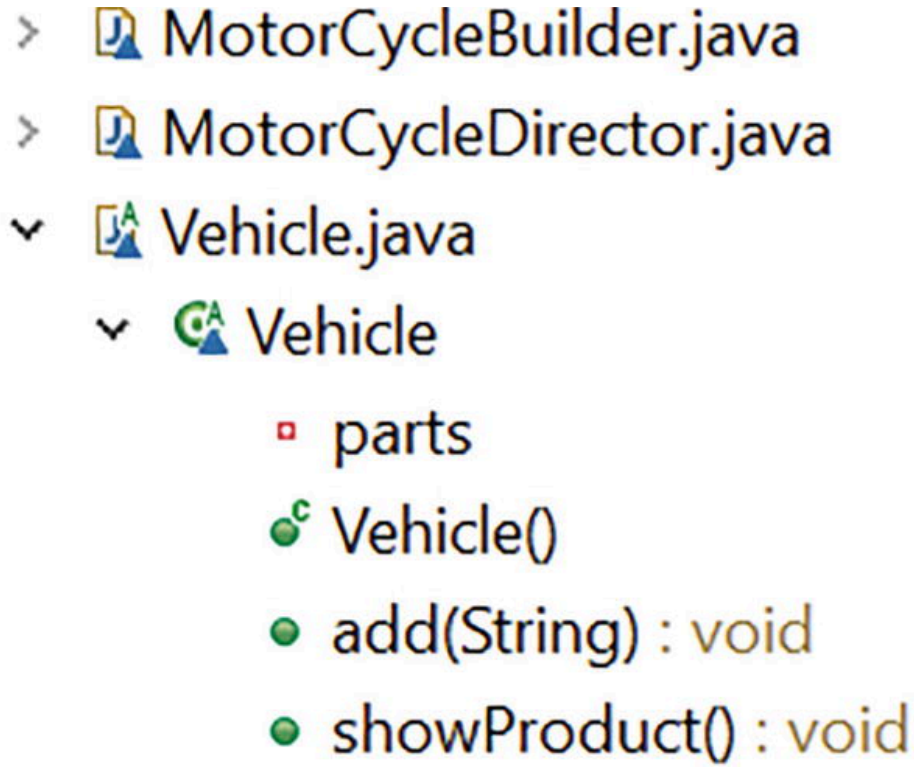


Figure 6-3 Package Explorer view

> **Note** This program has many parts. To make the diagram short and simple, I just expand `Car`, `CarBuilder`, and `CarDirector` along with the necessary parts. You can also refer to the class diagram (Figure 6-2) if you need it. I follow the same mechanism for other snapshots of the book, so when a snapshot is really big, I expand/show only the important parts and make it short.

Demonstration 1

Here's the complete implementation. All parts of the program are stored inside the package named `jdp3e.builder.implementation_1`.

> **Note** If you want to reuse some of these parts (e.g., `Vehicle.java`, `Car.java`, and `MotorCycle.java`), you can change their respective visibility to `public` and import these parts in a different demonstration. To execute a program, I put all these parts in the same folder, so I generally choose `package-private` visibility. This same comment applies to all programs in this book.

```
// Builder.java
// This is the common interface
interface Builder {
```

```
     void addBrandName();
     void buildBody();
     void insertWheels();
     // The following method is used to
     // retrieve the object that is constructed.
     Vehicle getVehicle();
}
// CarBuilder.java
// The CarBuilder builds cars.
class CarBuilder implements Builder {
     Car car;
     public CarBuilder() {
          car=new Car("Ford");
     }
     @Override
     public void addBrandName() {
          car.add(" Adding the car brand: " + car.brandName);
     }
     @Override
     public void buildBody() {



          car.add(" Making the car body.");
     }
     @Override
     public void insertWheels() {
          car.add(" 4 wheels are added to the car.");
     }
     @Override
     public Vehicle getVehicle() {
          return car;
     }
}
// MotorCycleBuilder.java
// The MotorCycleBuilder builds motorcycles.
class MotorCycleBuilder implements Builder {
     MotorCycle motorCycle;
     public MotorCycleBuilder() {
      motorCycle=new MotorCycle("Honda");
     }
     @Override
     public void addBrandName() {
      motorCycle.add(" Adding the brand name: " +
```

```
                        motorCycle.brandName);
     }
     @Override
     public void buildBody() {
      motorCycle.add(" Making the body of the motorcycle.");
     }
     @Override
     public void insertWheels() {
      motorCycle.add(" 2 wheels are added to the motorcycle.");
     }



     @Override
     public Vehicle getVehicle() {
          return motorCycle;
     }
}
// Vehicle.java
/**
 * The Vehicle class is used to create the products.
 * Making the class abstract, so that
 * you cannot instantiate from it directly.
 */
import java.util.LinkedList;
abstract class Vehicle {
     /*
      * You can use any data structure that you prefer.
      * I have used LinkedList<String> in this case.
      */
     private LinkedList<String> parts;
     public Vehicle() {
          parts = new LinkedList<String>();
     }
     public void add(String part) {
          // Adding parts
          parts.addLast(part);
     }
     public void showProduct() {
          System.out.println("These are the construction sequences:")
          for (String part : parts)
               System.out.println(part);
     }
}
```

```
// Car.java
class Car extends Vehicle{



     String brandName;
     public Car(String brandName) {
          this.brandName=brandName;
          System.out.println("\nWe are about to make a " +
                                brandName+ " car.");
     }
}
// MotorCycle.java
class MotorCycle extends Vehicle{
     String brandName;
     public MotorCycle(String brandName) {
          this.brandName=brandName;
          System.out.println("\nWe are about to make a " +
                                 brandName+ " motorcycle.");
     }
}
// Director.java
abstract class Director {
     // Director knows how to use/instruct the
     // builder to create a vehicle.
     public abstract Vehicle instruct(Builder builder);
}
// CarDirector.java
/**
 * The CarDirector directs the
 * car's instantiation steps




.
 */
class CarDirector extends Director {
     // The car director follows
     // its own sequence:
     // Make body-> add wheels->then add the brand name.
     public Vehicle instruct(Builder builder) {
          builder.buildBody();
          builder.insertWheels();
          builder.addBrandName();
```

```
          return builder.getVehicle();
     }
}
// MotorCycleDirector.java
/**
 *  The motorcycle director directs the
 *  motorcycle's instantiation steps.
 */
class MotorCycleDirector extends Director {
     // The motor cycle director follows
     // its own sequence:
     // Add brand name-> make body-> insert wheels.
     public Vehicle instruct(Builder builder) {
          builder.addBrandName();
          builder.buildBody();
          builder.insertWheels();
          return builder.getVehicle();
     }
}
// Client.java
class Client {
     public static void main(String[] args) {
         System.out.println("*** Builder Pattern Demonstration. ***")
         // Making a car



         Builder builder = new CarBuilder();
         Director director = new CarDirector();
         Vehicle vehicle=director.instruct(builder);
         vehicle.showProduct();
         // Making a motorcycle
         builder = new MotorCycleBuilder();
         director = new MotorCycleDirector();
         vehicle=director.instruct(builder);
         vehicle.showProduct();
     }
}
```

Output

Here’s the output:

```
*** Builder Pattern Demonstration. ***
We are about to make a Ford car




.
These are the construction sequences:
 Making the car body.
 4 wheels are added to the car.
 Adding the car brand: Ford
We are about to make a Honda motorcycle.
These are the construction sequences:
 Adding the brand name: Honda
 Making the body of the motorcycle.
 2 wheels are added to the motorcycle.
```

Q&A Session

6.1 What is the advantage of using a Builder pattern?

Here are some advantages:

- You direct the builder to build the objects step by step, and you promote encapsulation by hiding the details of the complex construction process. The director can retrieve the final product from the builder when the whole construction is over. In general, at a high level, you seem to have only one method that makes the complete product, but other internal methods are involved in the creation process. So, you have finer control over the construction process.
- Using this pattern, the same construction process can produce different products.
- In short, by changing the type of a builder, you change the internal representation of the product.

6.2 What are the drawbacks associated with a Builder pattern?

Here are some challenges:

- It is not suitable if you want to deal with mutable objects (which can be modified later).

- You may need to duplicate some portion of the code. These duplications may cause a performance impact in some contexts.
- To create more products, you need to create more concrete builders.

6.3 In demonstration 1, Builder is an interface. Could you use an abstract class instead of the interface in the illustration of this pattern?

Yes. You could use an abstract class instead of an interface in this example.

6.4 How do you decide whether to use an abstract class or an interface in an application?

If you want to have some centralized or default behaviors, an abstract class is a better choice. In those cases, you can provide some default implementation. On the other hand, the interface implementation starts from scratch and indicates some kind of rules/contracts such as what is to be done, but it does not enforce the "how" part upon you. Also, interfaces are preferred when you try to implement the concept of multiple inheritance.

Remember that if you need to add a new method in an interface, then you need to track down all the implementations of that interface, and you need to put the concrete implementation for that method in all those places. In such a case, an abstract class is a better choice because you can add a new method in an abstract class with a default implementation, and the existing code can run smoothly.

Java has taken special care on this last point. Java 8 introduced the use of the `default` keyword in the interface. From Java 8 onwards, you can prefix the word `default` before your intended method signature and provide a default implementation. Interface methods are public by default, so you do not need to mark them by the keyword `public`.

Here are the summarized suggestions from the ORACLE Java documentation (**https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html**). You should give preferences to abstract class for the following scenarios:

- You want to share code among multiple closely related classes.

- The classes that extend the abstract class can have many common methods or fields, or they require non-public access modifiers inside them.
- You want to use non-static or/and non-final fields that enable you to define methods that can access and modify the state of the object to which they belong.

On the other hand, you should give preferences to interfaces for these scenarios:

- You expect that several unrelated classes are going to implement your interface. For example, the interfaces `Comparable` and `Cloneable` can be implemented by many unrelated classes.
- You express your concern only to specify the behavior of a particular data type, but it does not matter how the implementer implements it.
- You want to use the concept of multiple inheritance in your application.

6.5 Why are you using a separate class for the director? You could use the client code to play the role of the director.

No one restricts you from doing that. In the preceding implementation, I wanted to separate this role from the client code. But in the upcoming demonstration, I use the client as a director.

6.6 What do you mean by client code?

A client is any class that uses another class (or interface). In the previous program, the class that contains the `main()` method is the client code. In most parts of the book, with the words *client code*, I mean the same.

6.7 You have talked several times about varying steps. I can see that the car director and the motorcycle director follow a different sequence of steps. Is this correct?

Yes. It is a good find. The upcoming implementation is even better. This demonstration uses method chaining to vary the steps inside the client code. It is very useful in this kind of design.

Alternative Implementation

Now I'll show you an alternative implementation. Here are the key characteristics of the modified implementation:

- The client code itself plays the role of a director in this implementation.
- This time I show you the use of method chaining to make a product with varying steps.
- Similar to the previous example, `Builder` represents the builder interface but the return type of the first three methods is changed (earlier it was void; now it is `Builder`). Notice the following interface with the changes in bold:

```
interface Builder {
     Builder addBrandName();
     Builder buildBody();
     Builder insertWheels();
     // The following method is used to
     // retrieve the object that is constructed.
     Vehicle getVehicle();
}
```

- Just like demonstration 1, the `CarBuilder` and `MotorCycleBuilder` classes implement all the methods defined in the interface. Here is a sample with the key changes in bold:

```
class CarBuilder implements Builder {
Car car;
public CarBuilder() {
car=new Car("Ford");}
@Override
public Builder addBrandName() {
     // Starting with brand name
     car.add(" Adding the car brand:" + car.brandName);
     return this;
}
@Override
public Builder buildBody() {
     car.add(" Making the car body.");
     return this;
}
public Builder insertWheels() {
```

```
     car.add(" 4 wheels are added to the car.");
     return this;
}
@Override
public Vehicle getVehicle() {
     return car;
 }
}
```

- Notice that these methods are similar to the previous demonstration but there is one major change: their return type is `Builder`. Since the return type is `Builder`, now you can apply **method chaining** . Before you assemble the parts (using the `getVehicle()` method) and display the complete product (using the `showProduct()` class),you can vary these steps. This is why, inside `main()`, you'll see code segments like the following:

```
// Construction Steps:
// Make body-> add wheels->then add the brand name.
       Vehicle vehicle=builder.buildBody()
              .insertWheels()
              .addBrandName()
              .getVehicle()
       vehicle.showProduct();
```

Since most of the parts are similar to the previous implementation, let's directly jump into the next demonstration.

Demonstration 2

Here is the alternative implementation for the Builder pattern. All the parts are stored inside package `jdp3e.builder.implementation_2`.

```
// Builder.java
import java.util.LinkedList;
// The common interface
interface Builder {
     Builder addBrandName();
     Builder buildBody();
     Builder insertWheels();
```

```
     // The following method is used to
     // retrieve the object that is constructed.
     Vehicle getVehicle();
}
// CarBuilder.java
// The CarBuilder builds cars.
class CarBuilder implements Builder {



     Car car;
     public CarBuilder() {
          car=new Car("Ford");
     }
     @Override
     public Builder addBrandName() {
          // Starting with brand name
          car.add(" Adding the car brand: " + car.brandName);
          return this;
     }
     @Override
     public Builder buildBody() {
          car.add(" Making the car body.");
          return this;
     }
     @Override
     public Builder insertWheels() {
          car.add(" 4 wheels are added to the car.");
          return this;
     }
     @Override
     public Vehicle getVehicle() {
          return car;
     }
}
// MotorCycleBuilder.java
// The MotorCycleBuilder builds motorcycles.
class MotorCycleBuilder implements Builder {
     MotorCycle motorCycle;
     public MotorCycleBuilder() {



          motorCycle=new MotorCycle("Honda");     }
```

```
     @Override
     public Builder addBrandName() {
          motorCycle.add(" Adding the brand name: " +
                         motorCycle.brandName);
          return this;
     }
     @Override
     public Builder buildBody() {
          motorCycle.add(" Making the body of the motorcycle.");
          return this;
     }
     @Override
     public Builder insertWheels() {
          motorCycle.add(" 2 wheels are added to the motorcycle.");
          return this;
     }
     @Override
     public Vehicle getVehicle() {
          return motorCycle;
     }
}
// Vehicle.java



/**
 * The Vehicle class is used to create the products.
 *  Making the class abstract, so that
 *  you cannot instantiate from it directly.
 */
abstract class Vehicle {
     /*
      * You can use any data structure that you prefer.
      * I have used LinkedList<String> in this case.
      */
     private LinkedList<String> parts;
     public Vehicle() {
          parts = new LinkedList<String>();
     }
     public void add(String part) {
          // Adding parts
          parts.addLast(part);
     }
     public void showProduct() {
```

```
          System.out.println("These are the construction sequences:")
          for (String part : parts)
               System.out.println(part);
     }
}
// Car.java
class Car extends Vehicle{
     String brandName;
     public Car(String brandName) {
          this.brandName=brandName;
          System.out.println("\nWe are about to make a " +
                              brandName+ " car.");
     }
}
// MotorCycle.java



class MotorCycle extends Vehicle{
     String brandName;
     public MotorCycle(String brandName) {
          this.brandName=brandName;
          System.out.println("\nWe are about to make a " +
                              brandName+ " motorcycle.");
     }
}
// Client.java
/**
 * The client is the director now.
 */
class Client {
 public static void main(String[] args) {
          System.out.println("*** Builder Pattern Demo2(Using
                              method chaining) ***");
          // Making a car
          Builder builder = new CarBuilder();
          // Construction Steps:
          // Make body-> add wheels->then add the brand name.
          Vehicle vehicle=builder.buildBody()
                     .insertWheels()
                     .addBrandName()
                     .getVehicle();
          vehicle.showProduct();
          // Making a motorcycle
```

```
            builder = new MotorCycleBuilder();
            // Add brand name-> make body-> insert wheels.
            vehicle=builder.addBrandName()
                   .buildBody()
                   .insertWheels()
                   .getVehicle();
            vehicle.showProduct();
      }
}
```

Output

Here's the output, which is the same except for the first line:

```
*** Builder Pattern Demo2(Using method chaining)***
We are about to make a Ford car.
These are the construction sequences:
 Making the car body.
 4 wheels are added to the car.
 Adding the car brand: Ford
We are about to make a Honda motorcycle.
These are the construction sequences:
 Adding the brand name: Honda
 Making the body of the motorcycle.
 2 wheels are added to the motorcycle.
```

Analysis

Examine the `main()` method closely. You can see that the director (client, in this example) creates two different products using the builders and each time it follows a different sequence of steps. Using this process, you can make an efficient and flexible application.

Q&A Session Continued

6.8 When should I consider using a Builder pattern?

As mentioned, if you need to make a complex object that involves various steps of the construction process and at the same time the products need to be immutable, the Builder pattern is a good choice.

> **Note** In the second edition of this book, I promoted immutability inside demonstration 2. There I used the `final` keyword for the concrete products. Also, there was a class called `ProductClass`. Inside this class, the attributes were marked with `private` keywords and there were no setter methods. Based on feedback, in this edition I show you two different demonstrations that are closely related. With some simple modifications, you can switch between them and understand the intent of the Builder pattern easily.

6.9 What is the key benefit associated with immutable objects?

Once constructed, they can be safely shared, and most importantly, they are thread-safe, so you save lots of synchronization costs in a multi-threaded environment.

In these examples, cars and motorcycles are the final products. Instead of using these products, I could use closely related products whose internal representations are quite different, such as sports cars and standard cars, or something similar. In fact, this pattern could be described using concrete products only. Also, to reduce the code size, I could use only one director to avoid the use of an abstract class. But I want you to think about this pattern from all possible angles. So, the overall code size is relatively big, but I hope that you have a clear understanding of this pattern now.

Previous chapter
< 5. Prototype Pattern

Next chapter
7. Singleton Pattern >